In [ ]:
```python
# Q1) Objective: Create a class and objects.
# P3Q1a) Create a Student class containing: Roll Number, Name, Marks.
# Create methods to: Accept  data, Display data, Calculate grade

class Student:
    def __init__(self,rollno,name):
        self.rollno = rollno
        self.name = name
        self.subjects = []
        self.marks = []

    def accept(self):
        self.rollno=input("Enter rollno:")
        self.name=input("Enter name:")
        no_of_subjects = int(input("Enter number of subjects:"))

        for sub in range(no_of_subjects):
            subject = input(f"Enter subject {sub + 1} name:")
            mark = float(input(f"Enter marks for {subject}:"))
            self.subjects.append(subject)
            self.marks.append(mark)

    def calculate_grade(self, mark):
        if mark >= 90:
            return 'A'
        elif mark >= 80:
            return 'B'
        elif mark >= 70:
            return 'C'
        elif mark >= 60:
            return 'D'
        else:
            return 'F'

    def display(self):
        print("Roll No:",self.rollno)
        print("Name:",self.name)
        for subject, mark in zip(self.subjects, self.marks):
            print("\nSubject:",subject)
            print("Marks:",mark)
            print("\nGrade:",self.calculate_grade(mark))

student1 = Student(0,"")
student1.accept()
student1.display()
```

```
Roll No: 101
Name: alice

Subject: math
Marks: 90.0

Grade: A

Subject: science
Marks: 80.0

Grade: B

Subject: english
Marks: 70.0

Grade: C
```

In [ ]:
```python
# P3Q1b) Create a product class with:
# Product name, Price, Quantity,
# Method to calculate total value
class product():
    def __init__(self,name,price,quantity):
        self.name=name
        self.price=price
        self.quantity=quantity

    def total_value(self):
        print(f"Total value of {self.name}: {self.price*self.quantity}")

p=product("laptop",10000,3)
p.total_value()
```

```
Total value of laptop: 30000
```

In [ ]:
```python
# P3Q2) Objective: Inheritance of multiple classes and objects
# Question : Create class Person with some attribute and method.
# Inherit Class Person in Student add some attribute and method.
# Inherit Class Student in Employee add some attribute and method.
# Create 2 objects of Person, 3 of students, and 5 of employees.
# Initialize parent attributes and
# Access the parent attribute and methods in child class.

class people():
    def __init__(self,name,address,email,gender,age):
        self.name = name
        self.address = address
        self.email = email
        self.gender = gender
        self.age = age

    def speak(self):
        print(f"{self.name} is speaking.")

    def walk(self):
        print(f"{self.name} is walking.")

    def breathe(self):
        print(f"{self.name} is breathing.")
```

```
    def age(self):
        print(f"{self.name} is aging.")

class student(people):
    def __init__(self, name, address, email, gender, age, studentid, prn,
                 course,last_degree, year_of_enrollment):
        super().__init__(name, address, email, gender, age)
        self.studentid = studentid
        self.prn = prn
        self.course = course
        self.last_degree = last_degree
        self.year_of_enrollment = year_of_enrollment

    def attend_exam(self):
        print(f"{self.name} is attending an exam.")

    def go_for_class(self):
        print(f"{self.name} is going for class.")

    def participate_in_events(self):
        print(f"{self.name} is participating in events.")

class employee(student):
    def __init__(self, name, address, email, gender, age, studentid, prn,
                 course, last_degree, year_of_enrollment, eid, department,
                 designation, salary):
        super().__init__(name, address, email, gender, age, studentid, prn,
                         course, last_degree, year_of_enrollment)
        self.eid = eid
        self.department = department
        self.designation = designation
        self.salary = salary

    def submit_report(self):
        print(f"{self.name} is submitting a report.")

    def calc_salary(self):
        print(f"{self.name} is calculating salary.")
        print(self.salary * 12)

people1 = people("Alice", "Goa", "alice@example.com", "Female", 25)
people2 = people("Bob", "Mumbai", "bob@example.com", "Male", 30)

student1 = student("Charlie", "Delhi", "charlie@example.com", "Male", 20,
                   1, 101, "Computer Science", "Bachelor", 2020)
student2 = student("David", "Kolkata", "david@example.com", "Male", 22,
                   2, 102, "Mechanical Engineering", "Bachelor", 2020)
student3 = student("Eve", "Chennai", "eve@example.com", "Female", 21,
                   3, 103, "Electrical Engineering", "Bachelor", 2020)

employee1 = employee("Frank", "Bangalore", "frank@example.com", "Male", 35,
                     4, 104, "Civil Engineering", "Master", 2020, 1001, "IT",
                     "Software Engineer", 50000)
employee2 = employee("Grace", "Hyderabad", "grace@example.com", "Female", 32,
                     5, 105, "Chemical Engineering", "Master", 2020, 1002,
                     "HR", "HR Manager", 60000)
employee3 = employee("Henry", "Pune", "henry@example.com", "Male", 38,
                     6, 106, "Biomedical Engineering", "Master", 2020, 1003,
                     "Finance", "Financial Analyst", 55000)
employee4 = employee("Ivy", "Jaipur", "ivy@example.com", "Female", 34,
```

```
                    7, 107, "Environmental Engineering", "Master", 2020, 1004,
                    "Marketing", "Marketing Manager", 65000)
employee5 = employee("Jack", "Lucknow", "jack@example.com", "Male", 40,
                    8, 108, "Nuclear Engineering", "Master", 2020, 1005,
                    "Operations", "Operations Manager", 70000)

student1.speak()
student1.go_for_class()
#student1.submit_report()

employee1.speak()
employee1.submit_report()
```

```
Charlie is speaking.
Charlie is going for class.
Frank is speaking.
Frank is submitting a report.
```

In [ ]:

```
import pandas as pd

# P3Q3) Create a E-commerce Analytics Pipeline:
# P3Q3a) Construct a transformation pipeline for e-commerce transactions.

transactions = pd.DataFrame({
    "transaction_id": [1, 2, 3, 4, 5],
    "customer": ["A", "B", "A", "C", "B"],
    "product": ["Laptop", "Mouse", "Keyboard", "Laptop", "Mouse"],
    "category": ["Electronics", "Accessories", "Accessories",
                 "Electronics", "Accessories"],
    "quantity": [1, 2, 1, 1, 3],
    "unit_price": [60000, 800, 1500, 60000, 800],
    "discount": [0.10, 0.05, 0, 0.15, 0.10]
})

# Transformation pipeline
ecommerce = (
    transactions
    .drop_duplicates()
    .assign(
        gross_value=lambda df: df["quantity"] * df["unit_price"],
        discount_value=lambda df:
            df["quantity"] * df["unit_price"] * df["discount"],
        net_value=lambda df: (
            df["quantity"] * df["unit_price"] * (1 - df["discount"])
        )
    )
)

print("Processed Transactions:")
display(ecommerce)

sales_summary = (
    ecommerce.groupby("category", as_index=False)
    .agg(
        total_orders=("transaction_id", "count"),
        total_quantity=("quantity", "sum"),
        total_sales=("net_value", "sum")
    )
    .sort_values("total_sales", ascending=False)
)
```

```
print("Sales Summary:")
display(sales_summary)
```

Processed Transactions:

	transaction_id	customer	product	category	quantity	unit_price	discount	gross_value	discount_
0	1	A	Laptop	Electronics	1	60000	0.10	60000	6
1	2	B	Mouse	Accessories	2	800	0.05	1600	
2	3	A	Keyboard	Accessories	1	1500	0.00	1500	
3	4	C	Laptop	Electronics	1	60000	0.15	60000	9
4	5	B	Mouse	Accessories	3	800	0.10	2400	

Sales Summary:

	category	total_orders	total_quantity	total_sales
1	Electronics	2	2	105000.0
0	Accessories	3	6	5180.0

In [ ]:
```
# P3Q3b) Design a data-wrangling workflow for university survey data.
survey = pd.DataFrame({
    "student_id": [101, 102, 103, 104, 105],
    "department": ["CS", "IT", "CS", "Mechanical", "IT"],
    "year": [1, 2, 3, 2, 1],
    "satisfaction": [4, 5, None, 3, 4],
    "study_hours": [5, 7, 6, None, 8],
    "recommend": ["Yes", "Yes", "No", "Yes", "Yes"]
})

# Clean, transform, and summarize survey data
survey_clean = survey.copy()

survey_clean["satisfaction"] = survey_clean["satisfaction"].fillna(
    survey_clean["satisfaction"].median())
survey_clean["study_hours"] = survey_clean["study_hours"].fillna(
    survey_clean["study_hours"].median())
survey_clean["satisfaction_level"] = survey_clean["satisfaction"].map(
    lambda score: "Satisfied" if score >= 4 else "Needs Improvement"
)

department_summary = (
    survey_clean.groupby("department", as_index=False)
    .agg(
        students=("student_id", "count"),
        average_satisfaction=("satisfaction", "mean"),
        average_study_hours=("study_hours", "mean"),
        recommendation_rate=("recommend", lambda x: (x == "Yes").mean() * 100)
    )
)

print("Cleaned Survey Data:")
display(survey_clean)
```

```
print("Department-wise Survey Summary:")
display(department_summary)
```

```
Cleaned Survey Data:
```

	student_id	department	year	satisfaction	study_hours	recommend	satisfaction_level
0	101	CS	1	4.0	5.0	Yes	Satisfied
1	102	IT	2	5.0	7.0	Yes	Satisfied
2	103	CS	3	4.0	6.0	No	Satisfied
3	104	Mechanical	2	3.0	6.5	Yes	Needs Improvement
4	105	IT	1	4.0	8.0	Yes	Satisfied

```
Department-wise Survey Summary:
```

	department	students	average_satisfaction	average_study_hours	recommendation_rate
0	CS	2	4.0	5.5	50.0
1	IT	2	4.5	7.5	100.0
2	Mechanical	1	3.0	6.5	100.0

In [ ]: